# Bridge Protocol App Summary

One-page repo-backed overview generated from the current codebase and README. Missing details are marked explicitly.

## What It Is

Bridge Protocol is a Next.js App Router prototype for civic issue discussion. The current app presents a consensus-ranked issue feed, a dedicated post composer, an issue detail view, a profile page, and a browser-local authentication flow. Core interactions run from client state and localStorage, with Supabase helpers and schema files included for a future backend migration.

## Who It's For

**Primary user/persona:** not explicitly documented in the repo.
**Best repo-backed inference:** people posting public-interest questions, tracking cross-cluster consensus, and reviewing counter-perspectives inside a mobile-first civic discussion product.

## What It Does

- Home feed ranks issue threads by consensus score and opposing-view support, with a prominent Bridge Meter card and issue summary metrics.
- Each issue card highlights category, state, urgency, vote totals, cluster mix, and a counter-perspective summary meant to surface disagreement constructively.
- Dedicated submit flow lets signed-in users publish new issue threads with category, state, title, and body fields.
- Auth modal supports local email/password demo accounts plus optional Google Identity sign-in when `NEXT_PUBLIC_GOOGLE_CLIENT_ID` is configured.
- Issue detail page expands the main thread into a richer discussion view with comments and consensus context.
- Sidebar UI exposes a lightweight Bridge Status concept with reputation, reliability, and social-cluster labeling directly inside the feed.
- Issue and auth updates are persisted in browser storage so feed changes survive refresh without a server roundtrip.

## How It Works

- **Framework:** Next.js App Router with React and TypeScript; Tailwind CSS powers styling and the repo includes shadcn-style UI primitives under `src/components/ui`.
- **Entry points:** `src/app/page.tsx` for the consensus feed, `src/app/submit/page.tsx` for thread creation, and `src/app/issues/[id]/page.tsx` for per-issue detail.
- **Auth flow:** `src/lib/auth.ts` stores accounts and the active session in localStorage and emits browser events consumed by `src/components/auth/AuthModal.tsx`.
- **Issue flow:** `src/lib/issues.ts` seeds initial issue threads, persists mutations in localStorage, and emits `ISSUES_EVENT` so the feed updates reactively.
- **Data model:** `IssuePost`, `IssueComment`, social clusters, and counter-perspectives live in `src/types/issue.ts`.
- **Storage model:** the current product is intentionally browser-local; no active backend integration is required for the app to run.

## How To Run

- Run `npm install`.
- Start development with `npm run dev`.
- Open `http://localhost:3000`.
- Use the home feed to browse consensus-ranked issues and `/submit` to create a new thread after signing in.
- Optional production check: `npx next build --webpack` or `npm run build`, then `npm start`.